

| COMPUTAÇÃO EM NUVEM

Prof. Armando Kolbe Junior

INTRODUÇÃO

Desenvolvimento e implementação de aplicações em nuvem



Crédito: Diyajyoti/Shutterstock.

A computação em nuvem revolucionou a forma como desenvolvemos e implementamos aplicações. Com a crescente demanda por soluções escaláveis e eficientes, compreender as nuances do desenvolvimento em nuvem tornou-se essencial para qualquer profissional de TI. Nesta abordagem, iremos explorar metodologias de desenvolvimento, arquitetura de microsserviços, computação *serverless* e ferramentas de desenvolvimento em nuvem. O objetivo é tentar fornecer uma base bem sólida para que você possa entender e aplicar esses conceitos em projetos reais, promovendo inovação e eficiência em sua organização.

TEMA 1 – METODOLOGIAS DE DESENVOLVIMENTO

O desenvolvimento de aplicações em nuvem exige uma abordagem que promova a agilidade, a colaboração e a automação. As metodologias mais comuns utilizadas no desenvolvimento em nuvem são DevOps, Agile, Integração Contínua (CI) e Entrega Contínua (CD), além de práticas como Infraestrutura como Código (IaC). A seguir, vamos abordar cada uma dessas metodologias de forma geral.

1.1 DevOps e práticas Agile

- **DevOps** é uma metodologia que combina desenvolvimento (Dev) e operações (Ops) para aumentar a eficiência e a colaboração entre equipes. A integração contínua e a entrega contínua (CI/CD), que veremos a seguir, são práticas centrais do DevOps, permitindo que o código seja testado e implantado de forma automática e frequente. Isso reduz o tempo de ciclo de desenvolvimento e melhora a qualidade do software (Azad; Hyrynsalmi, 2023).
- **Agile** é uma metodologia de desenvolvimento que enfatiza a flexibilidade, a colaboração e a entrega incremental de software. As práticas Agile, como Scrum e Kanban, promovem iterações curtas e feedback contínuo, permitindo que as equipes respondam rapidamente às mudanças nos requisitos do projeto (Almeida et al., 2015).

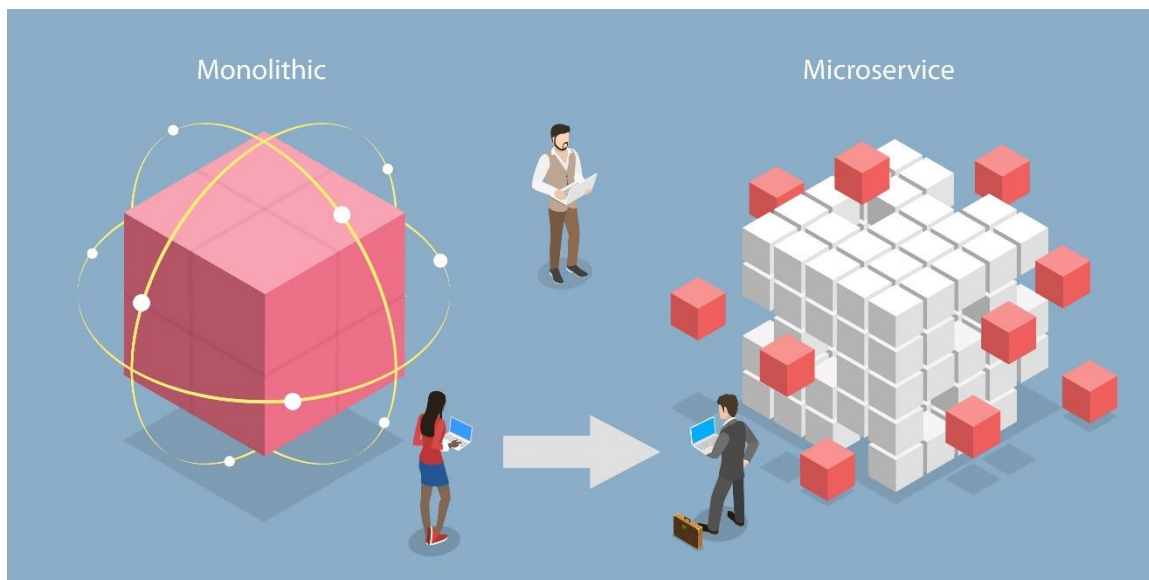
1.2 Integração e entrega contínuas (CI/CD)

- Integração Contínua (CI) é a prática de combinar código de todos os desenvolvedores em um repositório compartilhado várias vezes ao dia. Cada integração é verificada por meio de *builds* automáticos e testes, o que permite a detecção rápida de erros e conflitos (Objective, 2023; Silva & Huzita, 2021; Valadares Oliveira et al., 2021).
- Entrega Contínua (CD) vai além da CI, automatizando a implantação do código em ambientes de produção. Isso garante que o código esteja sempre em um estado pronto para ser lançado, permitindo a entrega rápida de novas funcionalidades e correções (Objective, 2023).

1.3 Infraestrutura como código (IaC)

Fernandes et al. (2022) afirmam que a Infraestrutura como Código (em inglês: *Infrastructure as Code – IaC*) é uma abordagem utilizada para definir uma infraestrutura de rede e computadores por meio de código-fonte, sendo tratada como uma aplicação de software. Os *scripts* IaC auxiliam os profissionais de TI a provisionar e gerenciar uma infraestrutura baseada em nuvem. Uma das ferramentas de IaC mais populares atualmente é o Docker, uma plataforma que automatiza a implantação de aplicações dentro de ambientes isolados denominados *containers* (Fernandes et al., 2022).

TEMA 2 – ARQUITETURA DE MICROSERVIÇOS



Crédito: TarikVision/Shutterstock.

Para Carmo et al. (2020) e Jeferson André Bigheti et al. (2018), a arquitetura de microserviços é uma abordagem de desenvolvimento de software que estrutura uma aplicação como um conjunto de serviços pequenos e independentes, cada um executando uma função específica e se comunicando por meio de APIs (Do Carmo et al., 2020; Jeferson André Bigheti et al., 2018).

2.1 Vantagens

Quadro 1 – Vantagens de uso da arquitetura de microserviços em nuvem

Vantagem	Descrição
Escalabilidade	Cada microserviço pode ser escalado de forma independente, permitindo que recursos sejam alocados exatamente onde são necessários. Isso é particularmente útil em ambientes de nuvem, onde a capacidade de escalabilidade dinâmica é uma das principais características.
Flexibilidade	Com os microserviços, diferentes equipes podem trabalhar em diferentes serviços de forma independente, utilizando tecnologias e linguagens de programação diferentes. Isso permite uma maior flexibilidade e agilidade no desenvolvimento.
Resiliência	A falha em um microserviço não afeta necessariamente os outros, aumentando a resiliência da aplicação como um todo. Na nuvem, isso é reforçado pela capacidade de distribuir serviços por várias regiões e zonas de disponibilidade.

Desempenho	A arquitetura de microsserviços pode melhorar o desempenho geral da aplicação, permitindo que partes da aplicação sejam desenvolvidas e otimizadas de forma independente.
Desenvolvimento e Implantação Contínua	A arquitetura de microsserviços facilita a integração e entrega contínuas (CI/CD), permitindo atualizações e lançamentos frequentes sem interrupção significativa dos serviços.
Gerenciamento de APIs	O gerenciamento eficaz de APIs é crucial em uma arquitetura de microsserviços. Ferramentas como <i>API Gateway</i> e <i>Service Mesh</i> ajudam a gerenciar, monitorar e proteger as APIs, facilitando a comunicação entre microsserviços e melhorando a segurança.

Fonte: elaborado por Armando Kolbe Junior com base em Carmo et al., (2020); Jeferson André Bigheti et al., 2018.

2.2 Ferramentas de desenvolvimento (*Spring Boot*, *Node.js*)

Supardi (2021) e Tilkov e Vinoski, (2010) apresentam algumas ferramentas populares para desenvolvimento de microsserviços:

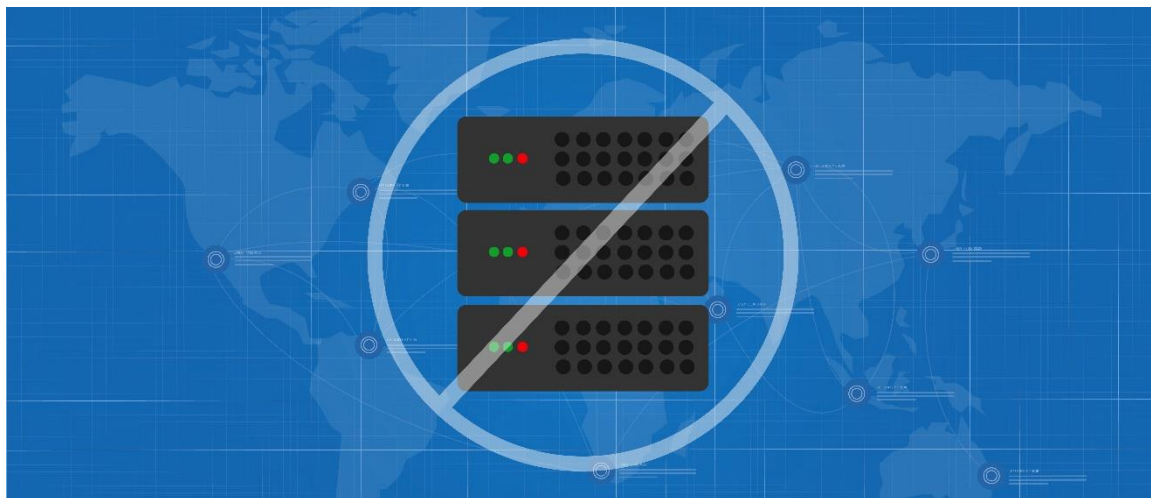
- **Spring Boot:** um *framework* baseado em Java que simplifica a criação de microsserviços (Ottinger; Lombardi, 2024).
- **Node.js:** uma plataforma de desenvolvimento que permite a criação de microsserviços rápidos e escaláveis usando JavaScript (Supardi, 2021; Tilkov; Vinoski, 2010).

2.3 Gestão de APIs (*API Gateway*, *Service Mesh*)

- **API Gateway:** atua como um ponto de entrada para as APIs, gerenciando o tráfego, segurança e roteamento das solicitações (Xu et al., 2019; Zuo et al., 2020).
- Para Duarte Maia e Figueiredo Correia (2022) e Yang et al., (2024), o **Service Mesh** facilita a comunicação entre microsserviços, fornecendo recursos como balanceamento de carga, segurança e monitoramento (Duarte Maia; Figueiredo Correia, 2022; Yang et al., 2024).

Se adotarem a arquitetura de microsserviços em nuvem, as organizações podem vir a obter uma maior agilidade, eficiência e resiliência em suas operações de TI, aproveitando ao máximo as vantagens oferecidas pelas plataformas de nuvem.

TEMA 3 – COMPUTAÇÃO *SERVERLESS*



Crédito: Bakhtiar Zein/Shutterstock.

A computação *serverless* é um modelo de execução em nuvem onde os provedores de serviços gerenciam automaticamente a infraestrutura necessária para executar o código do usuário, permitindo que os desenvolvedores se concentrem apenas na lógica da aplicação. Entre as vantagens, esse modelo oferece a escalabilidade automática, redução de custos e eliminação da necessidade de gerenciamento de servidores. Plataformas populares incluem AWS Lambda, Azure Functions e *Google Cloud Functions*. *Serverless* é ideal para aplicações que exigem alta flexibilidade e rapidez no desenvolvimento, sendo amplamente utilizado em cenários de microsserviços, processamento de eventos e aplicações de IoT.

3.1 Plataformas *serverless* (AWS Lambda, Azure Functions, Google Cloud Functions)

A computação *serverless* tem revolucionado a maneira como desenvolvedores criam, implantam e escalam aplicações. Nesse contexto, três das plataformas mais populares são AWS Lambda, Azure Functions e *Google Cloud Functions*. Cada uma dessas plataformas oferece um conjunto de ferramentas robustas para construir aplicações sem a necessidade de gerenciar servidores, permitindo que os desenvolvedores se concentrem apenas no código e na lógica de negócios.

3.2 AWS Lambda

De acordo com a AWS (2020), Giménez-Alventosa et al. (2019) e Villamizar et al. (2017), *AWS Lambda*, oferecida pela *Amazon Web Services*, é uma das primeiras e mais amplamente adotadas plataformas *serverless*. Com o *Lambda*, os desenvolvedores podem executar código em resposta a eventos sem provisionar ou gerenciar servidores.

A plataforma suporta uma variedade de linguagens de programação, incluindo Node.js, Python, Java e C#. *AWS Lambda* é frequentemente usada para tarefas de *backend*, como processar dados em tempo real, automatizar backups e executar scripts em resposta a mudanças em dados armazenados no Amazon S3 (AWS, 2020; Giménez-Alventosa et al., 2019; Villamizar et al., 2017).

3.3 Azure Functions

Azure Functions, da Microsoft, é outra plataforma *serverless* que permite aos desenvolvedores executar código em resposta a eventos desencadeados por diversos serviços da Azure ou por fontes externas. Suporta várias linguagens de programação, incluindo C#, JavaScript, Python e Java. *Azure Functions* é ideal para automação de fluxos de trabalho, integração de sistemas e tarefas de manutenção de aplicações. Uma característica distintiva é a integração profunda com o ecossistema de serviços da *Azure*, facilitando a criação de soluções complexas e escaláveis (Kettner; Geisler, 2022; Sawhney; Chanumolu, 2023).

3.4 Google Cloud Functions

Google Cloud Functions, parte do *Google Cloud Platform*, oferece um ambiente *serverless* para executar funções escritas em Node.js, Python e Go. É particularmente eficaz para construir microsserviços e APIs que escalam automaticamente conforme a demanda. A plataforma se destaca por sua integração nativa com outros serviços do *Google Cloud*, como Firebase, Pub/Sub e *Cloud Storage*, tornando-a uma escolha poderosa para desenvolvedores que já utilizam o ecossistema do Google (Malawski et al., 2020; Rajan, 2020).

3.5 Casos de uso e melhores práticas

Quadro 2 – Casos de uso

Caso de uso	Descrição
Processamento de dados em tempo real	Todas as três plataformas <i>serverless</i> são excelentes para processar grandes volumes de dados em tempo real. Isso inclui tarefas como análise de logs, processamento de <i>streams</i> de dados e manipulação de eventos de IoT.
Automação e integração	Empresas utilizam funções <i>serverless</i> para automatizar processos de negócios e integrar diferentes sistemas. Por exemplo, uma função pode ser acionada quando novos dados são inseridos em um banco de dados, realizando operações como validação de dados e notificação de equipes.
<i>Backends</i> para aplicações móveis e web	Plataformas <i>serverless</i> são ideais para criar <i>backends</i> escaláveis para aplicações móveis e web, oferecendo APIs que podem crescer conforme a demanda sem a necessidade de gerenciamento de infraestrutura.
Tarefas de manutenção e monitoramento	Funções <i>serverless</i> podem ser configuradas para executar periodicamente tarefas de manutenção, como backups e atualizações de sistemas, ou monitorar a saúde de outros serviços e recursos na nuvem.

Fonte: elaborado por Armando Kolbe Junior com base em AWS, 2020; Duarte Maia; Figueiredo Correia, 2022; Giménez-Alventosa et al., 2019; Villamizar et al., 2017; Yang et al., 2024.

Quadro 3 – Melhores práticas

Melhores práticas	Descrição
Design modular	Quebre sua aplicação em funções pequenas e modulares que realizam tarefas específicas. Isso facilita a manutenção e o escalonamento independente de cada função.
Gerenciamento de Estado	Como funções <i>serverless</i> são efêmeras e <i>stateless</i> por natureza, utilize serviços de armazenamento de estado, como bancos de dados ou caches, para gerenciar dados persistentes.
Monitoramento e <i>logging</i>	Implemente práticas robustas de monitoramento e <i>logging</i> para rastrear a <i>performance</i> e o comportamento das funções. Utilize serviços como AWS <i>CloudWatch</i> , Azure <i>Monitor</i> e Google <i>Stackdriver</i> para coletar e analisar logs e métricas.
Segurança	Aplique princípios de segurança, como o princípio do menor privilégio, garantindo que cada função tenha apenas as permissões necessárias para executar suas tarefas. Utilize ferramentas de segurança integradas nas plataformas para proteger dados e operações.
Custo-eficiência	Otimize o uso de recursos e a execução das funções para minimizar custos. Utilize a modelagem de custos das

plataformas para entender o impacto financeiro e ajustar configurações conforme necessário.

Fonte: elaborado por Armando Kolbe Junior com base em Giménez-Alventosa et al., 2019; Kettner; Geisler, 2022; Malawski et al., 2020; Rajan, 2020; Sawhney; Chanumolu, 2023; Yang et al., 2024).

Ao adotarmos plataformas *serverless* como *AWS Lambda*, *Azure Functions* e *Google Cloud Functions*, pode ser que consigamos trazer inúmeros benefícios para o desenvolvimento de aplicações modernas, desde a redução de custos operacionais até a melhoria da escalabilidade e flexibilidade. Seguir as melhores práticas poderá garantir que se aproveite ao máximo essas tecnologias avançadas, criando soluções eficientes e robustas para as necessidades dos negócios.

TEMA 4 – FERRAMENTAS DE DESENVOLVIMENTO EM NUVEM

Este tema aborda as principais ferramentas de desenvolvimento em nuvem, destacando SDKs e *frameworks*, como AWS SDK, Azure SDK e GCP SDK, que facilitam a criação e integração de aplicações em ambientes de nuvem. A Infraestrutura como Código (IaC) é explorada com ferramentas como Terraform e CloudFormation, que permitem o gerenciamento automatizado e escalável de recursos de infraestrutura. Além disso, as plataformas de desenvolvimento *low-code/no-code* são discutidas por sua capacidade de permitir que desenvolvedores e não desenvolvedores criem aplicações de maneira rápida e eficiente, promovendo inovação e agilidade nos processos de desenvolvimento.

4.1 SDKs e *frameworks* (AWS SDK, Azure SDK, GCP SDK)

De acordo com eficientes Falchuk et al. (2016) e Nguyen (2022), os SDKs (*Software Development Kits*) e *frameworks* fornecidos pelos principais provedores de nuvem, como AWS, Azure e *Google Cloud Platform* (GCP) são ferramentas essenciais para o desenvolvimento de aplicações na nuvem. O AWS SDK, por exemplo, oferece bibliotecas para várias linguagens de programação, facilitando a integração com os serviços da AWS, como S3, EC2 e DynamoDB.

Já o Azure SDK fornece um conjunto de bibliotecas para linguagens, como .NET, Java, Python e JavaScript, permitindo a criação e gerenciamento de recursos na plataforma Azure. Da mesma forma, o GCP SDK inclui ferramentas e bibliotecas para acessar serviços do *Google Cloud*, como *Google Cloud Storage*, **Compute Engine** e *BigQuery*. Esses SDKs e *frameworks* simplificam o desenvolvimento, gerenciamento e escalabilidade de aplicações na nuvem, fornecendo interfaces programáticas robustas e eficientes (Falchuk et al., 2016; Nguyen, 2022).

4.2 Infraestrutura como código (Terraform, CloudFormation)

A Infraestrutura como Código (IaC), já vista anteriormente, é uma abordagem que permite gerenciar e provisionar recursos de computação automaticamente por meio de arquivos de configuração legíveis por humanos, em vez de configurações físicas de hardware ou ferramentas de configuração interativas. O Terraform, desenvolvido pela HashiCorp, é uma das ferramentas mais populares de IaC, permitindo definir recursos de infraestrutura em arquivos de configuração declarativos. O Terraform suporta múltiplos provedores de nuvem, incluindo AWS, Azure e GCP, proporcionando uma solução agnóstica de nuvem.

O AWS CloudFormation, por outro lado, é uma ferramenta específica da AWS que permite aos usuários modelar e configurar seus serviços da AWS, utilizando templates em formato JSON ou YAML. Ambas as ferramentas ajudam a garantir que a infraestrutura seja configurada de forma consistente, reduzindo erros manuais e aumentando a eficiência do gerenciamento de recursos na nuvem (Bhat, 2022; Brikman, 2016; Carvalho; Araujo, 2020).

4.3 Plataformas de desenvolvimento *low-code/no-code*

As plataformas de desenvolvimento *low-code/no-code* estão revolucionando a maneira como as aplicações são construídas, permitindo que desenvolvedores e até mesmo usuários sem experiência em programação criem aplicações funcionais rapidamente. Essas plataformas fornecem interfaces visuais e ferramentas de arrastar e soltar que simplificam o processo de desenvolvimento, eliminando a necessidade de escrever código extensivo.

Exemplos notáveis incluem Microsoft Power Apps, Google AppSheet e OutSystems. Essas ferramentas são ideais para acelerar a criação de aplicações empresariais, automatizar processos e criar protótipos rápidos. Além disso, as plataformas *low-code/no-code* ajudam a preencher a lacuna entre as equipes de negócios e de TI, permitindo uma colaboração mais estreita e uma maior agilidade na resposta às demandas de mercado (Brikman, 2016; Silva et al., 2023).

TEMA 5 – SEGURANÇA NO DESENVOLVIMENTO DE APLICAÇÕES



Crédito: kinn_studio/Shutterstock.

A segurança no desenvolvimento de aplicações é um campo em constante evolução, com novas vulnerabilidades e ameaças surgindo constantemente. Acompanhar as atualizações do OWASP Top 10 e utilizar ferramentas de análise de segurança de código de última geração são práticas essenciais para garantir a proteção das aplicações.

5.1 OWASP Top 10

O OWASP Top 10 é uma lista com as dez vulnerabilidades mais críticas em aplicações web, elaborada pela Open Web Application Security Project (OWASP). Essas vulnerabilidades, quando exploradas, podem levar a sérias consequências, como vazamentos de dados, interrupções de serviços e perda financeira.

5.1.1 Mas como o OWASP Top 10 se relaciona com a computação em nuvem?

A computação em nuvem, com sua escalabilidade e flexibilidade, oferece um ambiente atrativo para o desenvolvimento e hospedagem de aplicações. No entanto, a migração para a nuvem não torna as aplicações automaticamente mais seguras. Na verdade, a complexidade dos ambientes em nuvem, com múltiplas camadas de infraestrutura e serviços, pode introduzir novos vetores de ataque (Glemser, 2022; Marchand-Melsom; Nguyen Mai, 2020; OWASP.acunetix, 2017).

5.2 As dez principais vulnerabilidades do OWASP Top 10, quando aplicadas ao contexto da nuvem, podem se manifestar de formas específicas

Quadro 4 – Vulnerabilidades do OWASP Top 10

Vulnerabilidade	Descrição
Injeção	Ataques de injeção em bancos de dados, comandos e outros contextos podem ser ainda mais perigosos em ambientes de nuvem, onde os dados podem estar mais centralizados e acessíveis.
Falhas criptográficas	A configuração incorreta de algoritmos e chaves criptográficas, comum em ambientes de nuvem, pode levar à interceptação e decodificação de dados sensíveis.
Injeção de comandos	A execução de comandos arbitrários em servidores em nuvem pode permitir que atacantes obtenham acesso não autorizado a sistemas e dados.
Design não seguro	A criação de APIs e interfaces web com funcionalidades inseguras pode expor as aplicações a diversos tipos de ataques.
Configuração incorreta de segurança	A configuração inadequada de serviços em nuvem, <i>firewalls</i> , grupos de segurança e outros controles de acesso pode criar brechas de segurança.
Componentes vulneráveis	A utilização de componentes de software com vulnerabilidades conhecidas pode comprometer a segurança de toda a aplicação.
Violação de níveis e controles de acesso	A concessão de privilégios excessivos a usuários e a falta de controle de acesso adequado podem permitir que atacantes acessem recursos sensíveis.
Falhas de validação de entrada	A falta de validação adequada das entradas de usuários pode permitir a injeção de código malicioso e a execução de comandos arbitrários.
Uso de componentes com vulnerabilidades conhecidas	A utilização de componentes de software com vulnerabilidades conhecidas pode comprometer a segurança de toda a aplicação.

Rastreamento de sessão

A gestão inadequada de sessões pode permitir que atacantes se façam passar por usuários legítimos.

Fonte: elaborado por Armando Kolbe Junior com base em Marchand-Melsom; Nguyen Mai, 2020; OWASP.acunetix, 2017.

Para mitigar essas vulnerabilidades em ambientes de nuvem, é fundamental realizar as ações descritas no quadro a seguir.

Quadro 5 – Ações para mitigar as vulnerabilidades do OWASP Top 10

Ação	Descrição
Adotar práticas de desenvolvimento seguro	Seguir as melhores práticas de desenvolvimento seguro, como a codificação segura, a validação de entradas e a gestão de erros.
Utilizar ferramentas de análise de segurança	Empregar ferramentas de análise de código estático e dinâmico para identificar vulnerabilidades.
Implementar controles de acesso	Controlar o acesso aos recursos da nuvem de forma granular, utilizando mecanismos como IAM (<i>Identity and Access Management</i>).
Manter os sistemas atualizados	Aplicar regularmente <i>patches</i> de segurança para corrigir vulnerabilidades em sistemas operacionais, aplicações e componentes.
Monitorar a segurança da nuvem	Implementar mecanismos de monitoramento contínuo para detectar atividades suspeitas e responder a incidentes de segurança.

Fonte: elaborado por Armando Kolbe Junior com base em Marchand-Melsom; Nguyen Mai, 2020; OWASP.acunetix, 2017.

Podemos, então, afirmar que o OWASP Top 10 serve como um guia valioso para desenvolvedores e profissionais de segurança que buscam proteger suas aplicações em nuvem. Ao entender as vulnerabilidades mais comuns e adotar as melhores práticas de segurança, é possível reduzir significativamente o risco de ataques e proteger os dados sensíveis da organização.

5.3 ODS relevantes

- **ODS 4: Educação de Qualidade:** a nuvem pode facilitar o acesso à educação de qualidade, proporcionando plataformas de ensino online e recursos educacionais digitais.
- **ODS 5: Igualdade de Gênero:** a nuvem pode ser utilizada para desenvolver ferramentas e plataformas que promovam a igualdade de gênero, como plataformas de empoderamento feminino e ferramentas de análise de dados com foco em gênero.

-
- **ODS 16: Paz, Justiça e Instituições Fortes:** a nuvem pode auxiliar na construção de instituições mais transparentes e eficientes, facilitando o acesso à informação e à participação cidadã.

REFERÊNCIAS

ALMEIDA, L. F. M. de et al. Avaliação do desempenho em agilidade na gestão de projetos. **Production**, 26(4), 2015. Disponível em: <<https://doi.org/10.1590/0103-6513.116213>>. Acesso em: 6 ago. 2024.

AWS. **AWS Lambda Developer Guide**. Aws, 2020.

AZAD, N.; HYRYNSALMI, S. DevOps critical success factors — A systematic literature review. **Information and Software Technology**, 157, 2023. Disponível em: <<https://doi.org/10.1016/j.infsof.2023.107150>>. Acesso em: 6 ago. 2024.

BHAT, K. Multicloud Orchestration using Terraform. **International Journal for Research in Applied Science and Engineering Technology**, 10(6), 2022. Disponível em: <<https://doi.org/10.22214/ijraset.2022.44760>>. Acesso em: 6 ago. 2024.

BIGHETI, J. A. et al. Proposta de arquitetura orientada a microsserviços para aplicações de internet das coisas industrial. **Proceedings XXII Congresso Brasileiro de Automática**. Disponível em: <<https://doi.org/10.20906/cps/cba2018-0971>>. Acesso em: 6 ago. 2024.

BRIKMAN, Y. **Why we use Terraform and not Chef, Puppet, Ansible, SaltStack, or CloudFormation**. 2016. Disponível em: <gruntwork.io>. Acesso em: 6 ago. 2024.

CARMO, A. P. do. et al. **Uso de um Espaço Inteligente Baseado em Visão Computacional para o Controle de Formação de Robôs Móveis**. 2020. Disponível em: <<https://doi.org/10.5753/sbcup.2020.11223>>. Acesso em: 6 ago. 2024.

CARVALHO, L. R. de; ARAUJO, A. P. F. de. Performance Comparison of Terraform and Cloudify as Multicloud Orchestrators. **Proceedings – 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing, CCGRID**, 2020. Disponível em: <<https://doi.org/10.1109/CCGrid49817.2020.00-55>>. Acesso em: 6 ago. 2024.

DUARTE MAIA, J. T.; FIGUEIREDO CORREIA, F. Service Mesh Patterns. **ACM International Conference Proceeding Series**, 2022. Disponível em: <<https://doi.org/10.1145/3551902.3551962>>. Acesso em: 6 ago. 2024.

FALCHUK, B. et al. Just-in-time reconnaissance and assistance for video game streams and players. **13th IEEE Annual Consumer Communications and Networking Conference, CCNC**, 2016. Disponível em: <<https://doi.org/10.1109/CCNC.2016.7444738>>. Acesso em: 6 ago. 2024.

FERNANDES, D. D.; AYRES, L. D. G.; SANT'ANNA, C. N. **Security Smells em Infraestrutura como Código utilizando Docker**. 2022. Disponível em: <<https://doi.org/10.5753/sbseg.2020.19261>>. Acesso em: 6 ago. 2024.

GIMÉNEZ-ALVENTOSA, V.; MOLTÓ, G.; CABALLER, M. A framework and a performance assessment for serverless MapReduce on AWS Lambda. **Future Generation Computer Systems**, 97, 2019. Disponível em: <<https://doi.org/10.1016/j.future.2019.02.057>>. Acesso em: 6 ago. 2024.

GLEMSER, T. OWASP Top 10. **Datenschutz Und Datensicherheit – DuD**, 46(11), 2022. Disponível em: <<https://doi.org/10.1007/s11623-022-1685-5>>. Acesso em: 6 ago. 2024.

KETTNER, B.; GEISLER, F. Azure Functions. **Pro Serverless Data Handling with Microsoft Azure**, 2022. Disponível em: <https://doi.org/10.1007/978-1-4842-8067-6_4>. Acesso em: 6 ago. 2024.

MALAWSKI, M. et al. Serverless execution of scientific workflows: Experiments with HyperFlow, AWS Lambda and Google Cloud Functions. **Future Generation Computer Systems**, 110, 2020. Disponível em: <<https://doi.org/10.1016/j.future.2017.10.029>>. Acesso em: 6 ago. 2024.

MARCHAND-MELSOM, A.; NGUYEN MAI, D. B. Automatic repair of OWASP Top 10 security vulnerabilities: A survey. **Proceedings, IEEE/ACM 42nd International Conference on Software Engineering Workshops, ICSEW**, 2020. Disponível em: <<https://doi.org/10.1145/3387940.3392200>>. Acesso em: 6 ago. 2024.

NGUYEN, N. Building an E-commerce Application Utilizing Firebase Cloud service. **Metropolia University of Applied Sciences (Issue Mobile Solutions)**, 2022.

OBJECTIVE. Saiba como as práticas de CI/CD podem alavancar as entregas de software na sua empresa. **Objective**, 28 jul. 2023. <<https://www.objective.com.br/insights/ci->

cd/#~:text=A%20Entrega%20Cont%C3%ADnua%20(CD)%2C,r%C3%A1pida
%2C%20eficiente%20e%20sem%20erros>. Acesso em: 6 ago. 2024.

OTTINGER, J. B.; LOMBARDI, A. Spring Boot. **Beginning Spring 6**, 2024. Disponível em: <https://doi.org/10.1007/978-1-4842-9833-6_7>. Acesso em: 6 ago. 2024.

OWASP.ACUNETIX. **OWASP top 10**, 1 (1), 2017. Disponível em:
<<https://owasp.org>>. Acesso em: 6 ago. 2024.

RAJAN, R. A. P. A review on serverless architectures-Function as a service (FaaS) in cloud computing. **Telkomnika (Telecommunication Computing Electronics and Control)**, 18(1), 2020. Disponível em: <<https://doi.org/10.12928/TELKOMNIKA.v18i1.12169>>. Acesso em: 6 ago. 2024.

SAWHNEY, R.; CHANUMOLU, K. Introduction to Azure Functions. **Beginning Azure Functions**, 2023. Disponível em: <https://doi.org/10.1007/978-1-4842-9203-7_1>. Acesso em: 6 ago. 2024.

SILVA, N. A. da; MURAKAMI, R. G.; COSTA, B. A. Mapeamento de territórios como as marcas podem se relacionar com seus públicos. **JNT – Facit Business and Technology Journal**, 1(43), 2023. Disponível em: <<https://doi.org/10.29327/2199304.1.43-34>>. Acesso em: 6 ago. 2024.

SILVA, G.; HUZITA, E. **DiSEN-CI**: Um Servidor de Integração Contínua de Modelos. Disponível em: <<https://doi.org/10.5753/sbsi.2013.5698>>. Acesso em: 6 ago. 2024.

SUPARDI, Y. Semua Bisa Menjadi Programmer JavaScript & Node.js. **Elax Media Komputindo**, 2021.

TILKOV, S.; VINOSKI, S. Node.js: Using JavaScript to build high-performance network programs. **IEEE Internet Computing**, 14(6), 2010. Disponível em: <<https://doi.org/10.1109/MIC.2010.145>>. Acesso em: 6 ago. 2024.

VALADARES OLIVEIRA, E. J. et al. Infraestrutura PAAS para um ambiente seguro e escalável da Plataforma Retratos da Atenção Primária à Saúde. **Revista Brasileira de Inovação Tecnológica Em Saúde**, 10(2), 2021. Disponível em: <<https://doi.org/10.18816/r-bits.v10i2.23901>>. Acesso em: 6 ago. 2024.

VILLAMIZAR, M. et al. Cost comparison of running web applications in the cloud using monolithic, microservice, and AWS Lambda architectures. **Service Oriented Computing and Applications**, 11(2), 2017. Disponível em: <<https://doi.org/10.1007/s11761-017-0208-y>>. Acesso em: 6 ago. 2024.

XU, R.; JIN, W.; KIM, D. Microservice security agent based on API gateway in edge computing. **Sensors (Switzerland)**, 19(22), 2019. Disponível em: <<https://doi.org/10.3390/s19224905>>. Acesso em: 6 ago. 2024.

YANG, W. Network shortcut in data plane of service mesh with eBPF. **Journal of Network and Computer Applications**, 222, 2024. Disponível em: <<https://doi.org/10.1016/j.jnca.2023.103805>>. Acesso em: 6 ago. 2024.

ZUO, X. et al. An API gateway design strategy optimized for persistence and coupling. **Advances in Engineering Software**, 148, 2020. Disponível em: <<https://doi.org/10.1016/j.advengsoft.2020.102878>>. Acesso em: 6 ago. 2024.